

THE PERCEPTRON

Definition

- The Perceptron is the simplest type of artificial neural network and one of the earliest algorithms designed for supervised binary classification (sorting data into exactly two classes).
- It was introduced by Frank Rosenblatt in 1958.
- *“The Perceptron looks at a set of numbers describing something, multiplies each number by how important it is, adds them up, and decides which of two classes the thing belongs to.”*

The Perceptron Formula

- The entire decision-making process described above can be written as a single mathematical formula:
- $y = f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$
- This is often written more compactly using summation notation:
- $y = f(\sum (w_i \cdot x_i) + b) \quad \text{for } i = 1 \text{ to } n$

Breakdown of the formula

Symbol	Name
$x_1, x_2, \dots, x_n$	Inputs / Features
$w_1, w_2, \dots, w_n$	Weights
b	Bias
$\sum (w_i \cdot x_i)$	Weighted Sum / Net Input
$f()$	Activation Function
y	Output

The Step (Activation) Function

- The classic Perceptron activation function is the step function, written as:
- $f(z) = 1$ if $z \geq 0$
- $f(z) = 0$ if $z < 0$

How the Perceptron Separates the Classes

The Decision Boundary

- The equation $\sum w_i x_i + b = 0$ is not just part of the formula — it is literally the equation of a line (when you have 2 features) or a flat plane (with more features). This line is called the decision boundary.

Separating the Class Fields in Your Dataset

- Before you can compute anything, you need to correctly identify which columns of your dataset are used for the calculation and which column is the answer you're checking against.

Example 1: AND Logic Gate

Row	x1 (feature)	x2 (feature)	t (target class)
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

Given weights (already trained): $w1 = 1$, $w2 = 1$, bias $b = -1.5$

Step 1: Write the formula for this problem:

$$z = (w1 \cdot x1) + (w2 \cdot x2) + b$$

$$z = (1)(x1) + (1)(x2) + (-1.5)$$

Step 2: Substitute each row's feature values and apply the step function:

Example 1: AND Logic Gate

Row	Computation: $z = x_1 + x_2 - 1.5$	z	Output $y = f(z)$	Matches t ?
1	$0 + 0 - 1.5$	-1.5	0	Yes
2	$0 + 1 - 1.5$	-0.5	0	Yes
3	$1 + 0 - 1.5$	-0.5	0	Yes
4	$1 + 1 - 1.5$	0.5	1	Yes

Given weights (already trained): $w_1 = 1$, $w_2 = 1$, bias $b = -1.5$

$$z = (w_1 \cdot x_1) + (w_2 \cdot x_2) + b$$

Step 3: Separate the classes.

The decision boundary is $x_1 + x_2 - 1.5 = 0$, or rewritten as $x_2 = 1.5 - x_1$. Only the point (1,1) lands above this line, correctly isolating the single Class 1 point from the three Class 0 points.

Example 1: AND Logic Gate

Row	Computation: $z = x_1 + x_2 - 1.5$	z	Output $y = f(z)$	Matches t ?
1	$0 + 0 - 1.5$	-1.5	0	Yes
2	$0 + 1 - 1.5$	-0.5	0	Yes
3	$1 + 0 - 1.5$	-0.5	0	Yes
4	$1 + 1 - 1.5$	0.5	1	Yes

Given weights (already trained): $w_1 = 1$, $w_2 = 1$, bias $b = -1.5$

$$z = (w_1 \cdot x_1) + (w_2 \cdot x_2) + b$$

Step 3: Separate the classes.

The decision boundary is $x_1 + x_2 - 1.5 = 0$, or rewritten as $x_2 = 1.5 - x_1$. Only the point (1,1) lands above this line, correctly isolating the single Class 1 point from the three Class 0 points.

Example 2: Training the Perceptron

Row	x1 (feature)	x2 (feature)	t (target class)	y/matches?	new y
1	0	0	0		
2	0	1	0		
3	1	0	0		
4	1	1	1		

Starting point: $w1 = 0$, $w2 = 0$, $b = 0$, learning rate η (eta) = 1

$$z = (w1 \cdot x1) + (w2 \cdot x2) + b$$

Learning rule:

➤ $wi_new = wi_old + \eta(t - y) \cdot xi$

➤ $b_new = b_old + \eta(t - y)$